

SYSTEM DESIGN — TOPIC 2

Client–Server Architecture

A detailed software-engineering interview guide

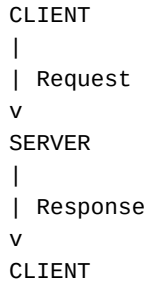
Goal: Build a strong mental model of how modern web applications communicate, how requests move through a system, and why architects introduce components such as load balancers, caches, API gateways, and databases.

Topics covered

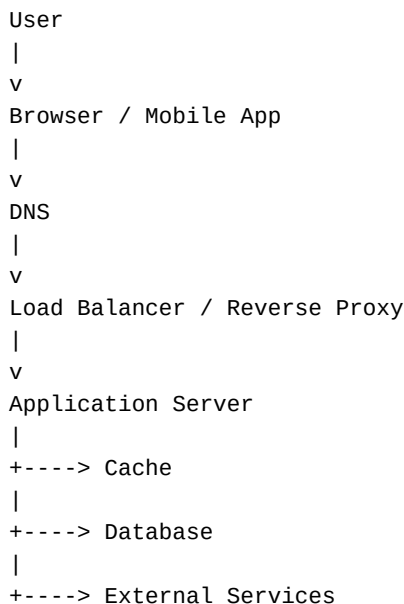
- Client, server, and network fundamentals
- Request/response lifecycle
- 2-tier, 3-tier, and N-tier architectures
- Stateless vs. stateful servers
- HTTP methods, status codes, headers, cookies and sessions
- REST API fundamentals
- Reverse proxies and API gateways
- Scaling application servers
- Common bottlenecks and failure modes
- Interview questions and a mini design exercise

1. The Core Mental Model

At the simplest level, a client–server system separates the software that requests a service from the software that provides it. The client is usually a browser, mobile application, desktop application, or another service. The server receives requests, performs work, accesses data or other services, and sends a response.



This is the foundation behind most web applications. A production system usually adds several layers around this basic model.



Interview principle: Do not begin a system-design interview by naming technologies. First identify the responsibilities of each component and the requirements that justify them.

2. Client vs. Server

Client

The client initiates communication. In a web application this may be a browser running JavaScript, a mobile app, or another backend service. The client is responsible for presenting information, collecting user input, storing some client-side state, and making network requests.

Server

The server provides functionality to clients. It validates requests, authenticates users, applies business rules, reads or writes data, calls other services, and constructs responses.

Why separate them?

Separation allows independent development and scaling. A backend can serve a web frontend, Android application, iOS application, and third-party integrations through APIs. It also allows application servers to be replicated horizontally.

3. What Happens When You Open a Website?

A simplified request path is:

1	User enters a URL	The browser interprets the URL and prepares to connect.
2	DNS lookup	The domain name is resolved to an IP address.
3	Connection setup	The client establishes a network connection; HTTPS also negotiates TLS.
4	HTTP request	The browser sends a request such as GET /products.
5	Edge/proxy layer	A CDN, reverse proxy, or load balancer may handle the request first.
6	Application server	Business logic processes the request.
7	Cache/database	The application may retrieve or persist data.
8	Response	The server returns a status code, headers, and optional body.
9	Rendering	The client displays or processes the response.

Key idea: A URL is not itself a server. It is an identifier that allows the client to locate a resource through DNS and then communicate using network protocols.

4. Architectural Tiers

2-Tier Architecture

```
CLIENT <-----> DATABASE / SERVER
```

A client communicates directly with a backend or database layer. It is simple, but direct database access from clients is usually inappropriate for modern internet-facing applications because it creates security, coupling, validation, and scaling problems.

3-Tier Architecture

```
PRESENTATION
|
v
APPLICATION / BUSINESS LOGIC
|
v
DATA / DATABASE
```

This is one of the most important architectures to understand. The presentation layer handles the user-facing interface, the application layer handles business rules, and the data layer manages persistence.

N-Tier Architecture

```
Client
|
v
CDN / Reverse Proxy
|
v
Load Balancer
|
v
API / Application Layer
|
+---- Cache
+---- Message Queue
+---- Database
+---- Search
+---- Object Storage
+---- External APIs
```

N-tier does not mean that every system needs many layers. Each additional layer introduces complexity. The architecture should be driven by requirements such as scale, security, team boundaries, reliability, and performance.

5. HTTP Fundamentals

HTTP is the application-layer protocol most commonly used for web APIs. A request contains a method, target path, headers, and sometimes a body.

```
GET /api/users/42 HTTP/1.1
Host: example.com
Accept: application/json
Authorization: Bearer <token>
```

Common HTTP methods

GET	Retrieve a resource	GET /users/42
POST	Create or trigger an operation	POST /users
PUT	Replace a resource	PUT /users/42
PATCH	Partially update a resource	PATCH /users/42
DELETE	Delete a resource	DELETE /users/42

Important status codes

200 OK	Request succeeded
201 Created	A resource was created
204 No Content	Succeeded without a response body
400 Bad Request	Client sent invalid input
401 Unauthorized	Authentication is missing/invalid
403 Forbidden	Identity is known but access is denied
404 Not Found	Resource/route was not found
409 Conflict	Request conflicts with current state
429 Too Many Requests	Rate limit exceeded
500 Internal Server Error	Unexpected server-side failure
502 Bad Gateway	Proxy received a bad upstream response
503 Service Unavailable	Service temporarily unable to handle request

6. Stateless vs. Stateful Servers

Stateless

```
Request 1 ---> Server A
Request 2 ---> Server B
Request 3 ---> Server C
```

Each request contains enough information for any healthy server to process it, or the required shared state is stored in a shared system such as a database or cache.

Stateless application servers are easier to scale horizontally because a load balancer can distribute requests across many instances.

Stateful

```
User ---> Server A
|
+-- session stored only on Server A

Next request ---> Server B
|
+-- cannot find the session
```

Stateful servers keep important session or connection state locally. This can make scaling and failover harder. Sticky sessions can partially address this, but they also create operational trade-offs.

Preferred interview answer: When practical, keep application servers stateless and place shared state in a durable/shared component. This is a design preference, not an absolute rule.

7. Sessions, Cookies and Authentication

A cookie is data stored by the client and sent with matching requests. Sessions allow a server to associate a client with server-side state. Authentication answers 'Who are you?' while authorization answers 'What are you allowed to do?'

```
Browser
|
| Cookie: session_id=abc123
v
Application Server
|
v
Session Store
|
v
User Identity
```

For scalable systems, session state is often stored in a shared database or distributed cache rather than only in one application process.

JWT-based alternative

```
Client ---> Authorization: Bearer <JWT> ---> API
|
+--> server validates token
```

JWTs can make certain APIs easier to scale because the server can validate a signed token without retrieving a session record for every request. However, token revocation, token size, key management, and security still require careful design.

8. REST APIs

REST is an architectural style commonly used for HTTP APIs. A good REST API models resources and uses HTTP semantics consistently.

```
GET /api/products
GET /api/products/123
POST /api/products
PATCH /api/products/123
DELETE /api/products/123
```

Good API design also considers pagination, filtering, sorting, validation, consistent error formats, versioning, authentication, authorization, idempotency, and rate limiting.

Pagination

```
GET /api/products?page=2&limit=20
```

For very large datasets, cursor-based pagination can be more stable and efficient than offset pagination, especially when records are frequently inserted or deleted.

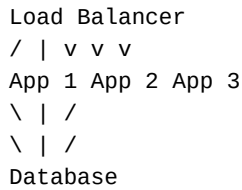
9. Reverse Proxy and API Gateway

A reverse proxy sits between clients and backend servers. It can terminate TLS, route requests, serve cached/static content, enforce limits, and hide internal infrastructure.

```
Internet
|
v
Reverse Proxy
|
+----> App 1
+----> App 2
+----> App 3
```

An API gateway is a more application-aware entry point that may provide authentication, routing, rate limiting, request transformation, aggregation, and observability. In microservice architectures it can provide a unified public API while services remain internal.

10. Load Balancing and Horizontal Scaling



If one application server can handle 500 requests/second and traffic grows beyond that, adding more instances can increase capacity. The load balancer distributes incoming requests.

Common strategies

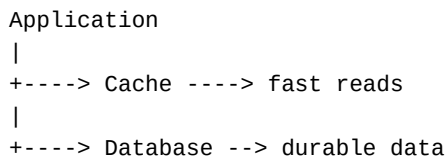
- **Round robin:** requests are distributed sequentially.
- **Weighted round robin:** stronger instances receive more traffic.
- **Least connections:** traffic is sent toward instances with fewer active connections.
- **Health checks:** unhealthy instances are removed from rotation.

Why statelessness helps

If App 1 fails, another server can handle the next request without losing critical application state. Shared databases, caches, queues, and object storage allow instances to remain replaceable.

11. Database as a System Component

The application server should usually not treat the database as an unlimited resource. Databases have constraints on CPU, memory, storage, connections, disk I/O, locks, and query execution.



Common scaling techniques include indexes, query optimization, connection pooling, read replicas, partitioning/sharding, caching, and workload separation.

Interview habit: Whenever you add a database, ask: What is the read/write ratio? How much data exists? What queries are common? What consistency is required? What happens when the database becomes unavailable?

12. Common Bottlenecks

CPU	Application logic is computationally expensive	Scale instances; optimize algorithms
Memory	Large in-memory objects or leaks	Optimize memory use; scale vertically/horizontally
Database	Slow queries, too many connections, lock contention	Indexes, caching, replicas, query optimization

Network	Large payloads or high traffic	Compression, CDN, pagination, efficient protocols
Single server	One machine receives all traffic	Load balancer + horizontal scaling
External API	Third-party service is slow/unavailable	Timeouts, retries, queues, caching, fallback

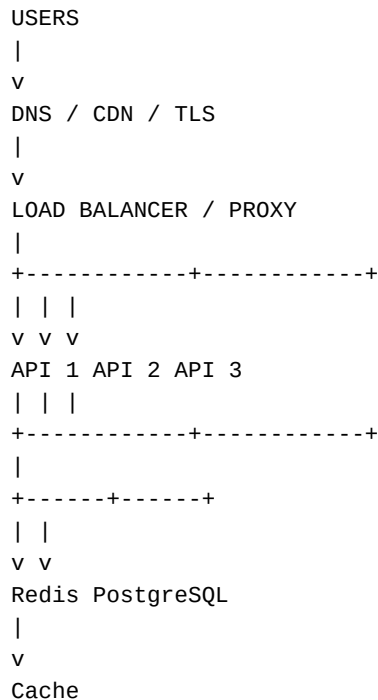
13. Failure Thinking

System design is largely about reasoning about failures. Assume that components will eventually fail.

- What if an application server crashes?
- What if the database becomes slow?
- What if a request times out?
- What if the same request is delivered twice?
- What if a downstream service is unavailable?
- What if traffic suddenly increases 10×?
- What if a network partition occurs?

Useful mechanisms include health checks, timeouts, retries with backoff, circuit breakers, queues, replication, failover, graceful degradation, idempotency, and monitoring.

14. Putting It Together: A Small Production Web System



Large/slow work:

API ---> Message Queue ---> Worker ---> Database / External Service

This architecture is intentionally generic. The important skill is understanding why each component exists. Redis is introduced if caching or shared ephemeral state is needed. A queue is introduced if work should be asynchronous or decoupled. Multiple API instances are introduced when one instance cannot provide enough capacity or availability.

Example request

GET /api/products/123

1. DNS resolves the domain.
2. CDN/proxy receives the request.
3. Load balancer selects a healthy API instance.
4. API validates/authenticates the request.
5. API checks Redis.
6. Cache hit: return product.
7. Cache miss: query PostgreSQL.
8. Store the result in Redis.
9. Return JSON response to client.

Notice how the architecture is not a collection of fashionable tools. Each step addresses a specific responsibility or performance/reliability requirement.

15. System Design Interview Questions

1. What is the difference between a client and a server?
2. Explain what happens when a user enters a URL in a browser.

3. What is the difference between 2-tier and 3-tier architecture?
4. Why are stateless servers easier to scale?
5. What is a reverse proxy?
6. What is the role of a load balancer?
7. What happens if one application server fails?
8. What is the difference between authentication and authorization?
9. When would you use cookies and sessions?
10. What are the common HTTP methods and status codes?
11. What makes an API RESTful?
12. How would you scale a backend from 1,000 to 1,000,000 requests/day?
13. Where would you introduce caching?
14. What would you do if the database became the bottleneck?
15. How would you handle a slow external API?

16. Mini Design Exercise

Problem: Design a simple job marketplace API where clients post jobs and freelancers browse and apply for them.

Start with requirements

Functional: user registration, job creation, job browsing, applications, messaging, notifications.

Non-functional: secure authentication, reasonable latency, availability, and ability to scale as users grow.

First architecture

```

Web / Mobile Clients
|
v
API
|
+----+----+
| |
Cache Database

```

Questions you should ask before adding complexity

- How many users and requests are expected?
- Is browsing read-heavy?
- How frequently are jobs created?
- Does messaging need real-time delivery?
- Can notifications be asynchronous?

- What data requires strong consistency?
- What happens if the database is unavailable?

17. Interview Checklist

- Clarify functional and non-functional requirements.
- Estimate scale before choosing architecture.
- Identify the main request flows.
- Start with a simple architecture.
- Explain why each component is needed.
- Think about bottlenecks and failure modes.
- Discuss database access patterns.
- Consider caching only where it solves a real problem.
- Keep services stateless when practical.
- Discuss security and observability.
- State trade-offs explicitly.

18. Key Takeaways

- **Client-server architecture** is the foundation of most networked applications.
- **3-tier architecture** separates presentation, business logic, and data responsibilities.
- **HTTP** provides the common request/response semantics used by web APIs.
- **Stateless application servers** make horizontal scaling and failover easier.
- **Load balancers** distribute traffic and can remove unhealthy instances.
- **Reverse proxies/API gateways** provide a controlled entry point into backend infrastructure.
- **Databases are bottlenecks** unless data access is designed carefully.
- **System design is trade-off analysis**, not technology-name memorization.

Next Topic

Topic 3 — Networking, HTTP, HTTPS, DNS and the Request Lifecycle in Depth.

We will go beneath the client-server abstraction and trace a request through DNS resolution, TCP/TLS, HTTP, proxies, connection reuse, latency, timeouts, and failure scenarios. These concepts are extremely valuable in Software Engineering interviews.